

Non-standard bytecode with non-contiguous slots #1462

Closed

While migrating my [Luon programming language](#) to the most recent LuaJIT 2.1 version (commit [18b087c](#) March 30, 2026), I hit a reproducible segfault which doesn't appear in LuaJIT 2.0, and which goes away if I switch off the JIT in version 2.1.

I tested on Linux x86_64, Debian Bookworm, using GCC.

The segfault happens in `lj_snap_restore` (via `lj_snap_replay -> snapshot_slots`) when running bytecode with deep call chains and `__index` metamethod dispatch. The crash happens reliably where callee functions use more local slots than the number of results returned.

Possible explanation: when `lj_record_ret` processes a return to a frame that is part of the trace (`J->framedepth > 0`), it copies the results into the caller's slots and adjusts `J->maxslot`, `J->baseslot`, and `J->base`. But it does not seem to clear the callee's old slots above the result area. If the callee used more slots than `nresults`, the old TRefs remain in `J->slot[]`. A subsequent CALL at a higher slot position then brings these stale entries into `rec_check_slots`'s validation range, causing a frame depth or a type mismatch. The same issue seems to exist in the continuation frame return path (`frame_iscont`), which is reached via metamethods like `__index`. The metamethod's slots above the new `J->maxslot` are not cleared either.

This doesn't surface with the standard Lua compiler because it seems to allocate registers sequentially and CALLs tend to sit at the top of the used range, so subsequent instructions naturally overwrite the stale slots. In constrast, my Luon bytecode generator allocates registers non-sequentially.

To reproduce, build the [LnRuntime2.pro Qt5 project](#) (follow the instructions in the Readme), and then run `luon <path to Luon directory>/testcases/Smalltalk80/Smalltalk.lnpro`. The segfault occurs immediately on start, after the Smalltalk display is drawn, or the latest when you move the mouse cursor into the window.

The following fix removes the segfault from my applications tested so far (I'm not sure if it covers all possible cases):

```
--- a/src/lj_record.c
+++ b/src/lj_record.c
@@ -979,6 +979,8 @@
     }
     lj_snap_add(J);
 }
+
+ {
+   BCReg callee_ms = J->maxslot;
+   for (i = 0; i < nresults; i++) /* Adjust results. */
+     J->base[i-1-LJ_FR2] = i < gotresults ? J->base[rbase+i] : TREF_NIL;
+   J->maxslot = cbase+(BCReg)nresults;
@@ -987,6 +989,11 @@
     lj_assertJ(J->baseslot > cbase+1+LJ_FR2, "bad baseslot for return");
     J->baseslot -= cbase+1+LJ_FR2;
     J->base -= cbase+1+LJ_FR2;
+
+   /* Clear stale callee frame/local slots above the result area. */
+   { BCReg j;
+   for (j = J->maxslot; j < cbase+1+LJ_FR2+callee_ms; j++)
+     J->base[j] = 0;
+   }
+   } else if (J->parent == 0 && J->exitno == 0 &&
+     !bc_isret(bc_op(J->cur.startins))) {
@@ -1008,9 +1015,11 @@
     memmove(J->base + cbase, J->base-1-LJ_FR2, sizeof(TRef)*nresults);
     memset(J->base-1-LJ_FR2, 0, sizeof(TRef)*(cbase+1+LJ_FR2));
  }
+ }
+ } else if (frame_iscont(frame)) { /* Return to continuation frame. */
+   ASMFunction cont = frame_contf(frame);
+   BCReg cbase = (BCReg)frame_delta(frame);
+   BCReg mm_ms = J->maxslot;
+   if ((J->framedepth -= 2) < 0)
+     lj_trace_err(J, LJ_TRERR_NYIRETL);
+   J->baseslot -= (BCReg)cbase;
@@ -1060,6 +1069,11 @@
     lj_assertJ(cont == lj_cont_condf || cont == lj_cont_condt,
       "bad continuation type");
  }
+
+   /* Clear stale continuation + metamethod frame slots. */
+   { BCReg j;
+   for (j = J->maxslot; j < cbase+mm_ms; j++)
+     J->base[j] = 0;
+   }
+   } else {
+     lj_trace_err(J, LJ_TRERR_NYIRETL);
+   }
```

MikePall

changed the title ~~Segfault because of stale JIT slot TRefs after CALL return in lj_snap_restore~~ Non-standard bytecode with non-contiguous slots
49 minutes ago

MikePall

added **invalid** 41 minutes ago

This doesn't surface with the standard Lua compiler because it seems to allocate registers sequentially and CALLs tend to sit at the top of the used range, so subsequent instructions naturally overwrite the stale slots. In constrast, my Luon bytecode generator allocates registers non-sequentially.

There's your problem: the JIT compiler relies on this property of the generated bytecode. And other parts of the VM, too, e.g. the GC.

The LuaJIT bytecode is an *internal* VM-specific format and has many implicit constraints. The VM is free to make assumptions about its construction, because the bytecode is generated by the VM itself.

The bytecode may only be used as an intermediate format for (unmodified) storage. It saves a bit of space and load time, but that's about it.

I have repeatedly stated that using the bytecode as a target ISA is not a good idea. This is hard to get right, very fragile and *will* break on every version.

And while we are at it: please stop publicly slandering LuaJIT in other forums, just because you had a bad experience using it as a target VM for a very different language with a clear impedance mismatch. This was a bad idea to begin with. You are NOT helping open source by publicly disparaging other open source projects.

1

MikePall

closed this as completed 41 minutes ago

please stop publicly slandering LuaJIT in other forums,

It's yourself who started this with publishing the bytecode format (attached). There was nowhere a mention that it should not be used as a target. Now claiming that is a bit cheap. It took me more than a day to nail this down, so you could at least integrate the fix.

[LuaJIT 2.0 Bytecode Instructions.pdf](#)

EDIT: I always publicly admired you and your work and helped to promote it. I never "slandered" LuaJIT. You will understand that your unexpected reaction here has taken me by surprise. LuaJIT is much too good to be limited to just one language. And obviously you had the same opinion when you documented the bytecode.

Add a comment

Write

Preview

H B I ≡ <> 🔗 ⋮ ≡ ≡ @ ↗ ↻ ↶

Use Markdown to format your comment

Reopen issue

Comment

Assignees

No one assigned

Labels

invalid

Type

No type

Fields

No fields configured for issues without a type.

Give feedback